

Proyecto 1 – 2026 – Gestión de riesgos, excepciones y ALU multiciclo en MIPS

Polo Gracia, Gonzalo

lunes, 6 de abril de 2026

Índice

Introducción.....	3
Verificación de los requisitos funcionales.....	4
RF1 JAL y RET.....	4
RF2 Anticipación de operandos.....	6
RF3 Riesgos de datos.....	8
RF4 Riesgos de control.....	10
RF5 Riesgos estructurales.....	13
RF6 Cambio a modo excepción.....	14
RF7 Retorno a modo usuario RTE.....	17
RF8 ALU multiciclo.....	18
RF9 Contadores.....	19
Cuantificación de horas dedicadas.....	20
Conclusiones y autoevaluación.....	21

Introducción

Este documento recoge el trabajo realizado para completar el primer proyecto, “Gestión de riesgos, excepciones y ALU multiciclo en MIPS”, de la asignatura AOC2. En él, se detallan los objetivos establecidos, la metodología seguida, los resultados obtenidos y las conclusiones finales extraídas del trabajo realizado.

El propósito principal de este proyecto es la aplicación de los conceptos de diseño sobre el procesador MIPS segmentado en un entorno VHDL. Para ello, se ha desarrollado la lógica de la Unidad de Anticipación (UA), la Unidad de Detención (UD), la implementación de nuevas instrucciones como JAL, RET y RTE, y se han gestionado el control de riesgos y las transiciones entre modo usuario y modo excepción. Además, se ha integrado una unidad ALU vectorial con soporte para operaciones multiciclo y se han implementado contadores de eventos para medir el rendimiento del sistema.

Cada uno de los apartados se relaciona con un requisito clave del proyecto, donde se incluye una breve descripción del mismo, un resumen de la implementación y un conjunto de pruebas para verificar el correcto funcionamiento.

Finalmente, se destacan los objetivos alcanzados en este proyecto junto con los conocimientos adquiridos durante la realización del mismo.

Verificación de los requisitos funcionales

A continuación, se demuestra el correcto funcionamiento de cada uno de los requisitos funcionales.

RF1 JAL y RET

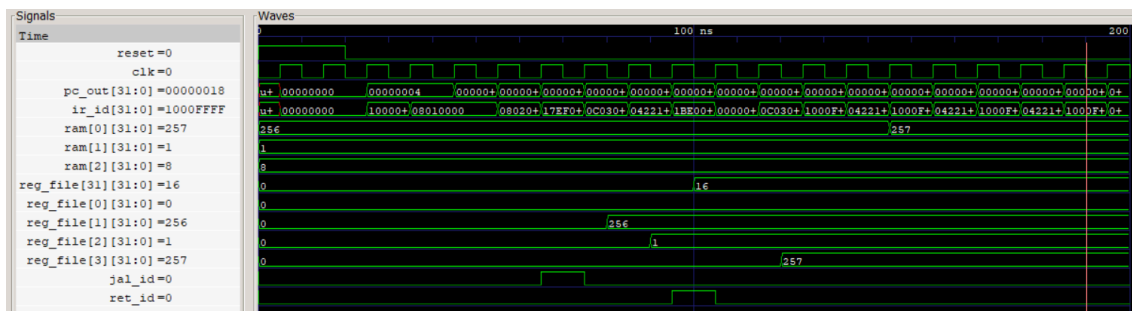
Descripción: Inclusión de las instrucciones ya trabajadas JAL (salto con enlace) y RET (retorno). Estas instrucciones permiten la ejecución de subrutinas mediante el almacenamiento y recuperación de la dirección de retorno al programa principal.

¿Cómo? En la Unidad de Control, para JAL, se han activado las señales jal y RegWrite, ya que esta instrucción debe escribir la dirección de retorno en el banco de registros; y para RET, se ha activado la señal ret, ya que se debe identificar el salto hacia la dirección contenida en un registro.

En el MIPS, se ha modificado el multiplexor de entrada al PC, PC_in. Para RET, se selecciona el valor BusA cuando la señal RET_ID está activa, y para JAL, se utiliza la dirección de salto Dirsalto_ID cuando la señal salto_tomado está activa. Además, las diferentes señales que se han definido para JAL y RET son extendidas a lo largo de las distintas etapas hasta llegar a la etapa WB. Destacar también que en esta última etapa se ha modificado el multiplexor 4 a 1 para que permita seleccionar el valor de PC4_WB como el dato a almacenar.

Verificación: testbench utilizado: JAL RET. Se han probado las siguientes instrucciones:

10000000	RESET	BEQ R0, R0, INI
08010000	INI	LW R1, 0(R0)
08020004		LW R2, 4(R0)
17EF0002		JAL R31, ADD_JAL
0C030000		SW R3, 0(R0)
1000FFFF	END	BEQ R0, R0, END
04221800	ADD_JAL	ADD R3, R1, R2
1BE00000		RET R31



JAL realiza el salto correctamente y almacena la dirección de retorno en el registro indicado y, tras ejecutar el RET, el programa continúa en la instrucción que se había quedado antes del salto.

RF2 Anticipación de operandos

Descripción: El procesador es capaz de anticipar los operandos para las instrucciones LW, SW y ARIT a distancia 1 (siempre que el dato a anticipar no venga de un LW) y 2.

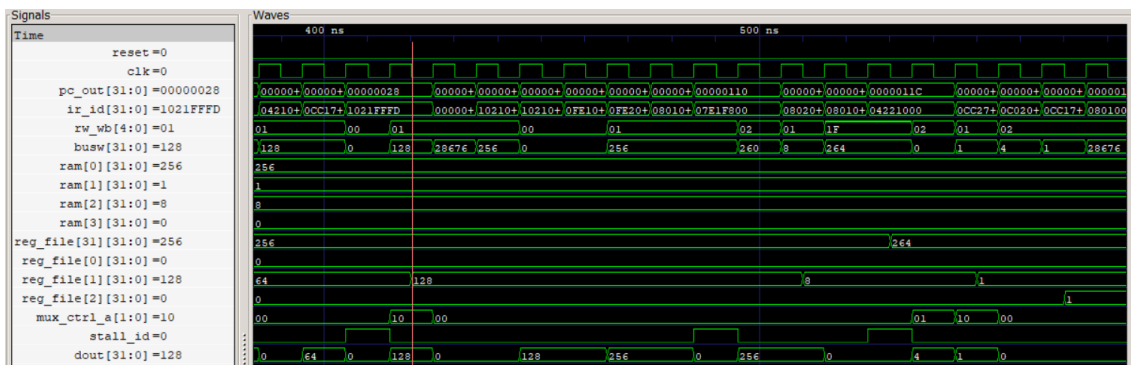
¿Cómo? La Unidad de Anticipación vigila qué registros necesitan los operandos de la ALU y los compara con lo que se escribe en las etapas siguientes y, dependiendo de esto, las señales MUX_ctrl_A y MUX_ctrl_B controlan los multiplexores Mux_A y Mux_B. Seleccionarán la entrada 0 si el dato viene del banco de registros, es decir, no hay anticipación; la entrada 1 si el dato viene de ALU_out_MEM, es decir, es el resultado que acaba de calcular en la ALU la instrucción anterior; o la entrada 2 si el dato viene de busW, es decir, es el dato que está listo para escribirse en la etapa WB.

De esta manera, si tenemos un ADD que escribe un resultado en un registro (etapa MEM) y la siguiente instrucción lo necesita (etapa EX), la señal Corto_A_Mem se pondrá a '1', ya que Reg_Rs_EX será igual a RW_MEM, por lo que MUX_ctrl_A será '01' y Mux_A elegirá ALU_out_MEM para que la siguiente instrucción reciba el valor antes de ser escrito en el registro.

En el Banco_EX_MEM el uso de Mux_B_out permite que, al hacer un SW, el valor del registro utilizado pueda ser anticipado y pasado a la etapa MEM para ser enviado a la memoria.

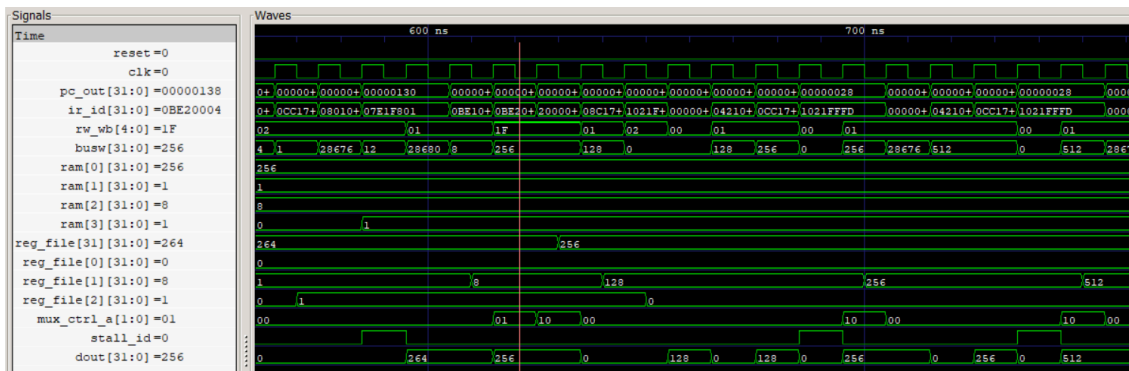
Verificación: testbench utilizado: Test IRQs. Se han probado varios casos:

En un momento dado se pasa de LW R1, 8(R0) a ADD R31, R1, R31.



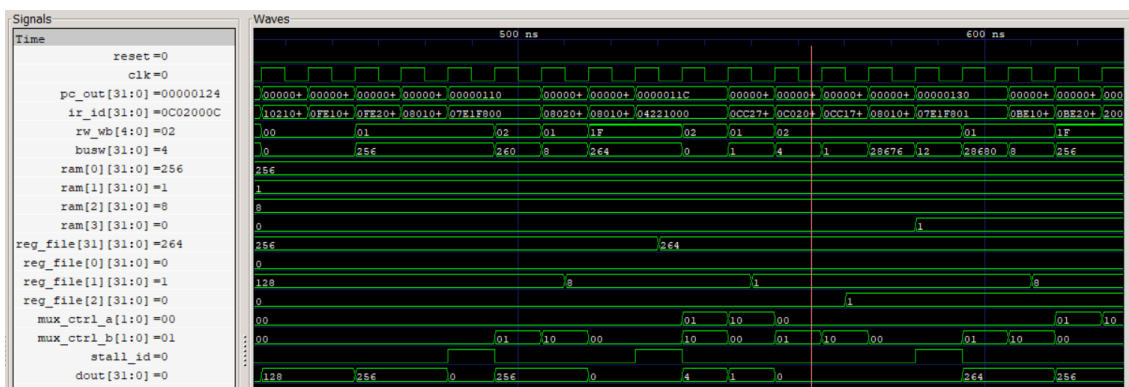
La señal stall_id sube a '1'. Esto quiere decir que ADD está en la etapa ID y detecta que R1 será escrito por un LW que todavía se encuentra en la etapa EX. En ese momento, mux_ctrl_a toma el valor '10' y la Unidad de Anticipación detecta que el valor de R1 ya ha llegado a la etapa WB, y el multiplexor de la ALU seleccionará bus directamente, permitiendo que el ADD use el valor cargado de memoria sin esperar a que se escriba en el banco de registros.

Un poco más adelante, se pasa de SUB R31, R31, R1 a LW R1, 0(R31):



En este caso, la señal mux_ctrl_a vale '01', lo que significa que se ha detectado una dependencia a distancia 1. La instrucción LW está en la etapa EX y SUB está en la etapa MEM, por lo que el valor que sale del multiplexor será igual al resultado que SUB acaba de generar, antes de que sea guardado en el banco de registros.

Un poco más adelante, se pasa de ADD R2, R1, R2 a SW R2, 7004(R6):



Mientras SW está en la etapa de ejecución, la señal mux_ctrl_b vale '01', lo que quiere decir que la Unidad de Anticipación detecta que el dato que SW quiere guardar es el mismo que ADD está terminando de procesar en la etapa MEM, permitiendo capturarlo correctamente para posteriormente ser escrito en memoria.

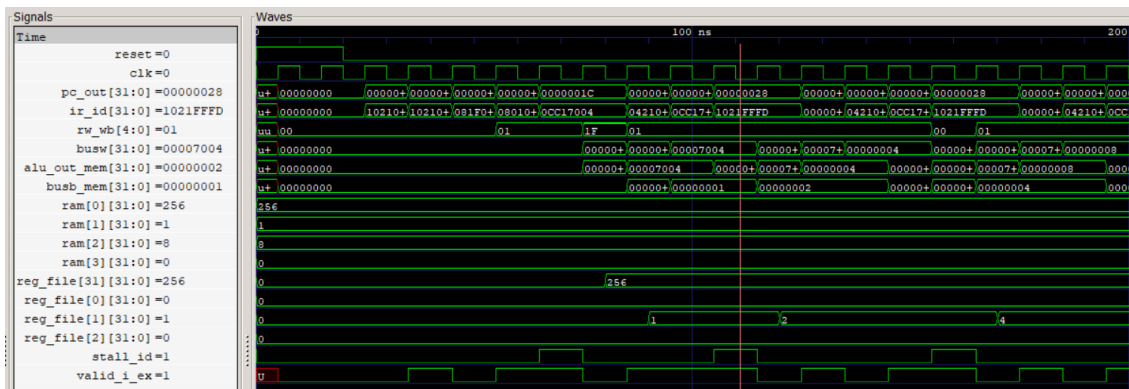
RF3 Riesgos de datos

Descripción: El procesador es capaz de detener la ejecución para evitar los riesgos causados por las dependencias en instrucciones lw-uso (distancia 1), beq o WRO (distancias 1 o 2).

¿Cómo? Si la Unidad de Detención detecta una dependencia que no se puede resolver (por ejemplo, un dato que aún no ha salido de la memoria) activa la señal stall_id. Esto implica congelar el PC y el registro de la etapa ID y evitar que las instrucciones en la etapa EX no realicen escrituras incorrectas.

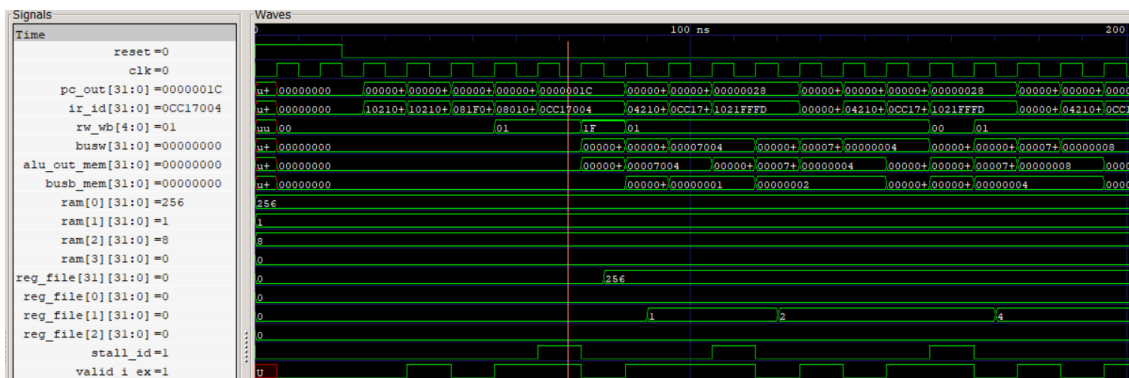
Verificación: testbench utilizado: Test IRQs. Se han probado varios casos:

En un momento dado, hay un ADD R1, R1, R1 y dos instrucciones después BEQ R1, R1, main:



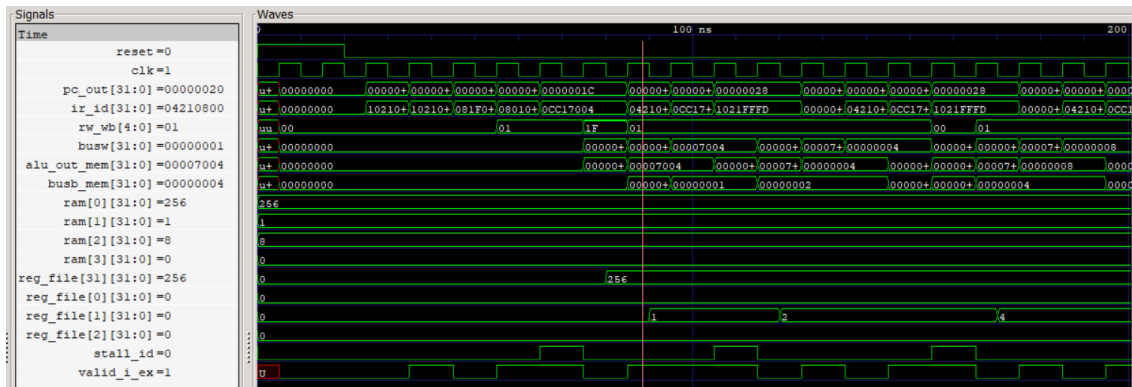
Se puede observar que la Unidad de Detención activa stall_id justo cuando el BEQ entra en la etapa ID. Esto detiene el avance del PC durante un ciclo, permitiendo que el dato del ADD llegue a la etapa MEM, permitiendo al BEQ realizar la comparación. Además, posteriormente la señal valid_i_ex pasa a valer '0' para indicar que la etapa EX se queda vacía a la espera de recibir el dato correcto.

En otro momento, hay un LW R1, 8(R0) y a continuación un SW R1, 7004(R6):



Al llegar al SW, se activa la señal stall_id y se congela el PC durante un ciclo ante esta dependencia. La señal valid_i_ex impide que el SW progrese con un dato erróneo, por lo que se retrasa la ejecución hasta que el dato de LW esté disponible en la etapa WB.

En otro momento, hay un SW R1, 7004(R6) y a continuación un ADD R1, R1, R1:



Se observa que cuando el ADD está en la etapa ID, la señal `stall_id` se mantiene en '0' y la señal `valid_i_ex` vale '1', confirmando que no se congela el PC, ya que SW no modifica el banco de registros. Por lo tanto, el ADD puede entrar sin problemas en la etapa EX y mantener el flujo de instrucciones.

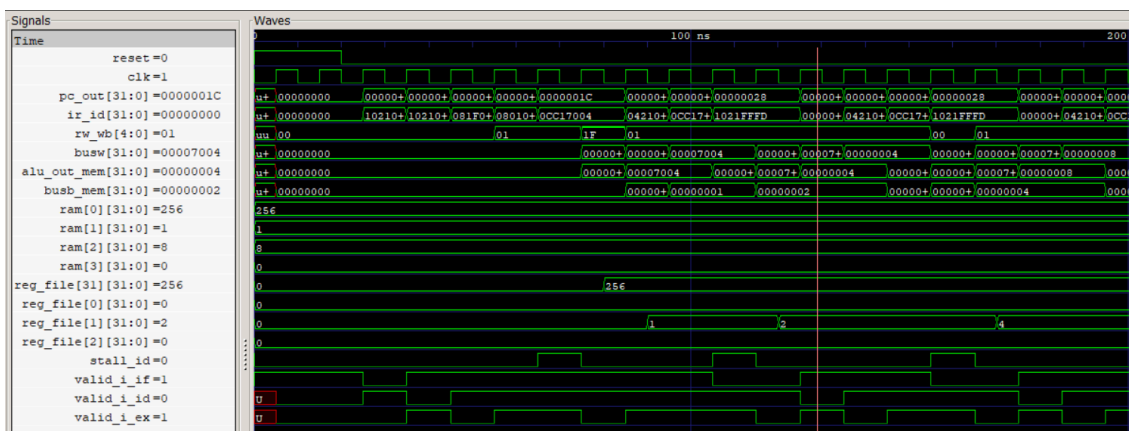
RF4 Riesgos de control

Descripción: El procesador es capaz de eliminar los riesgos de control causados por las instrucciones de salto: BEQ y RTE.

¿Cómo? Las instrucciones de salto se evalúan en la etapa ID y se decide si el salto se toma o no. Si es efectivo, el PC carga la dirección de destino. Dado que la instrucción que seguía al salto ya había entrado en la etapa IF, el procesador la anula poniendo a 0 el bit de validez, introduciendo un hueco cada vez que se toma un salto, garantizando que únicamente se ejecutan las instrucciones de la ruta correcta.

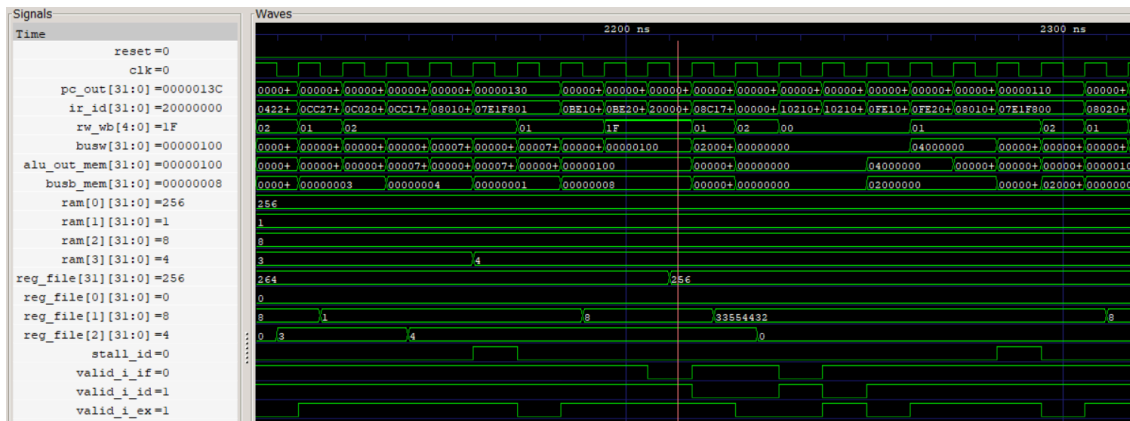
Verificación: testbench utilizado: Test IRQs. Se han probado varios casos:

En un momento dado, se realiza un BEQ R1, R1, main:



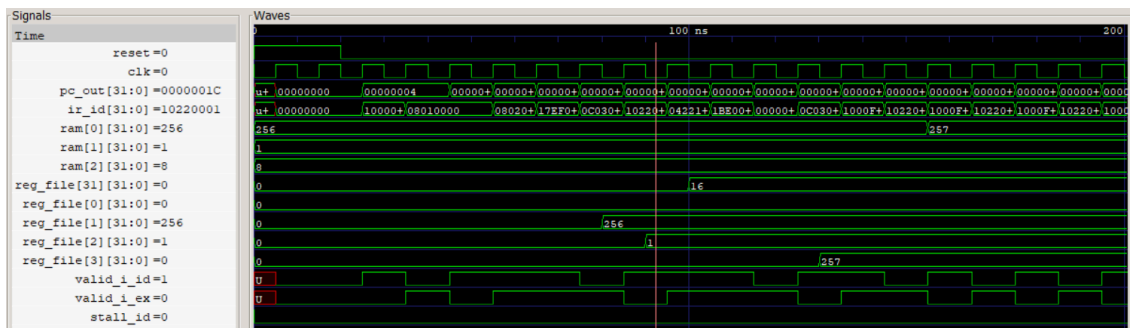
Se puede observar un ciclo donde valid_i_id se pone a '0', anulando la instrucción que se había cargado erróneamente debido al flujo secuencial. Simultáneamente, valid_i_if vuelve a '1', por lo que la primera instrucción del destino del salto está siendo cargada. La señal valid_i_ex está activa debido a que el BEQ acaba de pasar a la etapa EX. En este momento, el PC ya ha cargado la dirección de la siguiente instrucción a ejecutar, que es la del destino del salto.

Cuando se recibe una interrupción, para volver a la instrucción que se interrumpió, se ejecuta RTE:



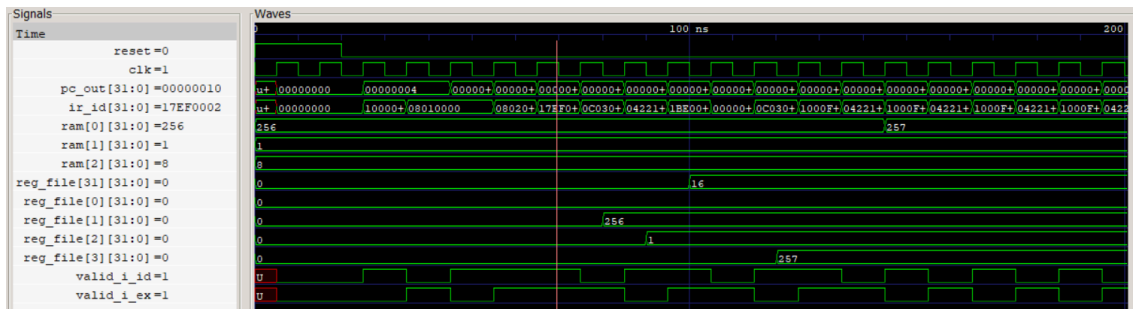
La señal valid_i_if pasa a '0', es decir, se detiene la búsqueda de la siguiente instrucción secuencial. La señal valid_i_id pasa a '0', es decir, se invalida la instrucción que estaba entrando en decodificación. Tras el ciclo de RTE, el PC deja de apuntar a la zona de la interrupción y vuelve a la dirección donde se había quedado en el programa principal.

Para hacer una prueba con un salto que no se tome, se ha añadido al testbench JAL RET utilizado previamente una instrucción de salto BEQ previa al ADD donde no se cumple la condición:



Se puede observar que en el momento de la instrucción BEQ la señal stall_id se mantiene en '0', es decir, el procesador no se detiene. La señal valid_i_id se mantiene en '1', y el PC pasa a la siguiente dirección sin realizar ningún salto.

Para verificar JAL y RET, se ha hecho una prueba con el testbench JAL RET mencionado anteriormente:



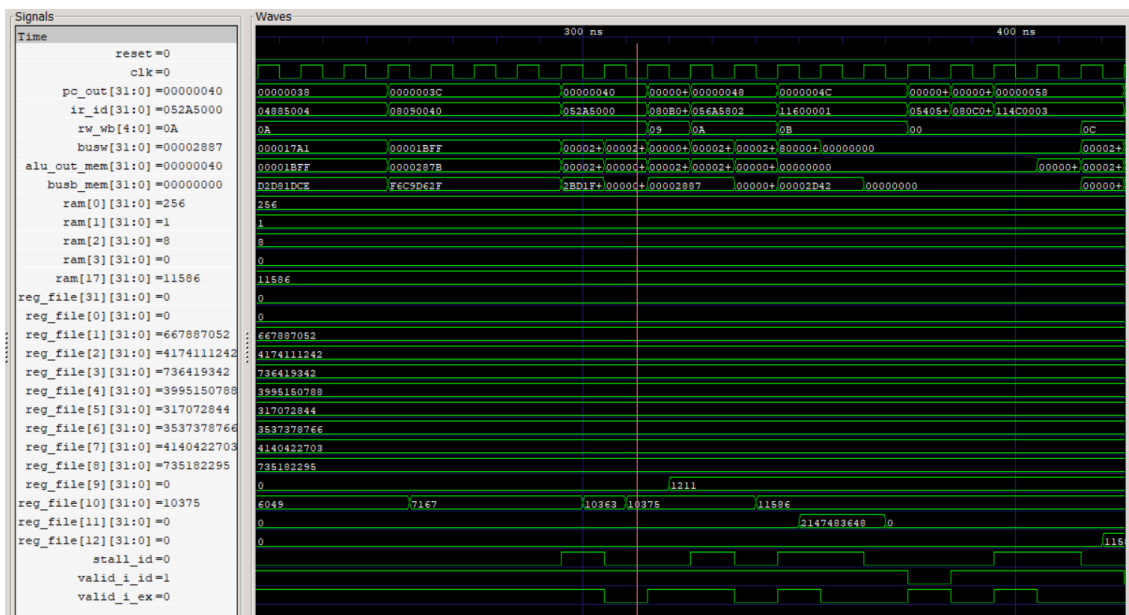
En ambas instrucciones se observa que, en el ciclo siguiente a su ejecución, la señal `valid_i_id` pasa a valer '0', invalidando la instrucción que se encontraba en la etapa IF, ya que el flujo debe volver atrás.

RF5 Riesgos estructurales

Descripción: El procesador es capaz de detener la ejecución para evitar riesgos estructurales causados por las instrucciones MAC y MAC_ini.

¿Cómo? Cuando una de estas instrucciones entra en la etapa de ejecución, la Unidad de Control detecta que la etapa EX estará ocupada durante unos ciclos adicionales. Entonces, activa la señal `stall_id`, impide que el PC avance y la instrucción se mantiene en la etapa EX durante el tiempo necesario.

Verificación: testbench utilizado: Delayed SYSTEM MAC (sin nops):



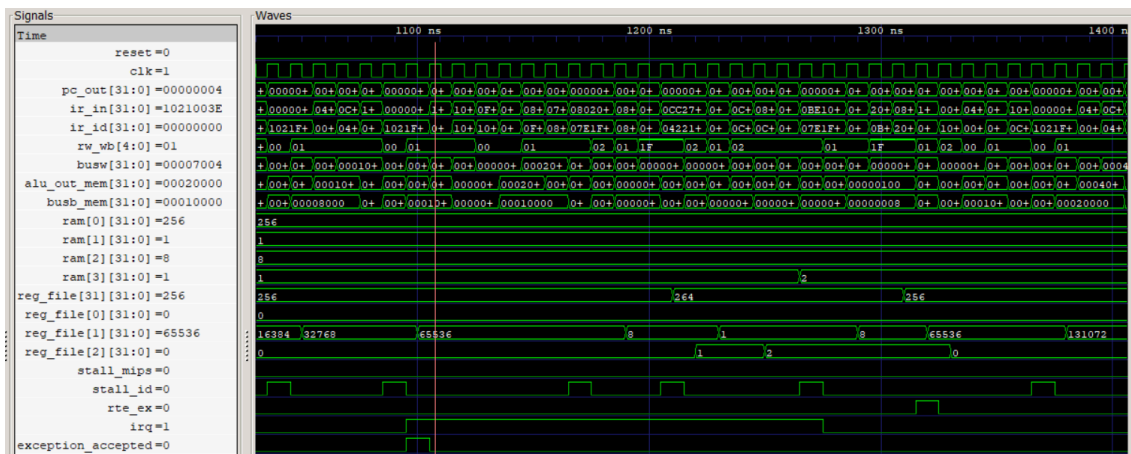
Se observa que justo cuando aparece el `MAC_ini`, la señal `stall_id` sube a '1' y el PC se queda congelado mientras se ejecuta dicha instrucción. La señal `valid_i_id` vale '1' ya que la siguiente instrucción es válida, pero justo después la señal `valid_i_ex` vale '0' ya que la instrucción `MAC_ini` continúa ejecutándose, y esto junto con `stall_id` mantiene retenida la siguiente instrucción momentáneamente. Exactamente lo mismo sucede un poco más adelante con la instrucción `MAC`. Además, en el registro R10 se observan los valores acumulados, coincidiendo el valor final con el resultado esperado, realizando los cálculos correctamente a pesar de las paradas.

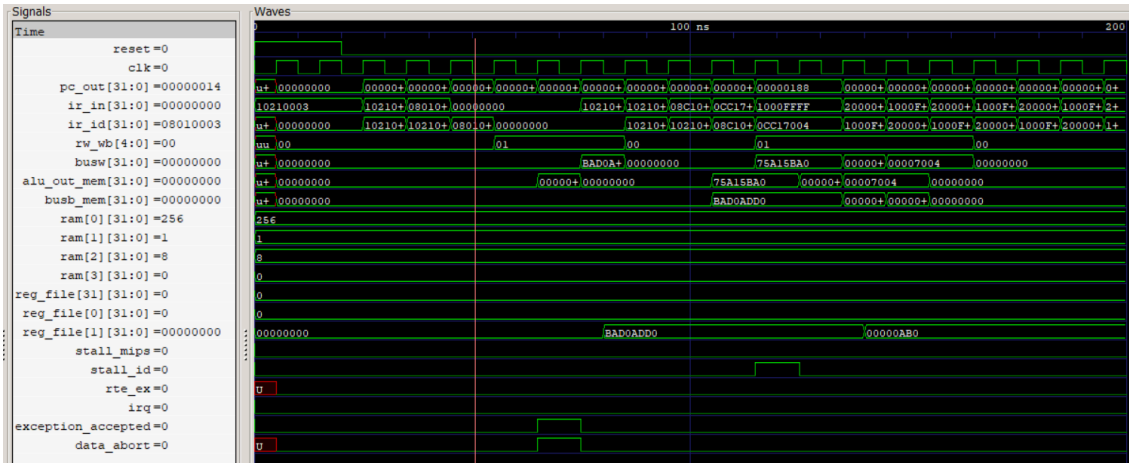
RF6 Cambio a modo excepción

Descripción: Si el procesador está en modo usuario y recibe IRQ, ABORT o UNDEF pasa al modo correspondiente y ejecuta la rutina que indica la tabla de vectores de excepción. Si está en modo excepción ignora las señales hasta volver a modo usuario.

¿Cómo? Al aceptar una excepción, se fuerza el PC a una dirección fija según el tipo de evento: 0x04 para IRQ, 0x08 para Data Abort o 0x0C para Undefined. En estas direcciones, una instrucción BEQ redirige el flujo hacia la rutina de servicio correspondiente y las instrucciones que iban a continuación no se ejecutan. Para salvar el estado del programa se utiliza el registro R31 como puntero de pila y se permitirá el retorno posterior mediante la instrucción RTE.

Verificación: testbenchs utilizados: Test IRQs para IRQ, Data Abort para DAbort y Undef para Undefined:





En este ejemplo se intenta acceder a una dirección de memoria no alineada. Esto es detectado y se activa la señal `data_abort` y, justo a la vez, la señal `exception_accepted`. Se fuerza al PC a cambiar al valor `0x8`, que es la dirección que en este caso está destinada a las excepciones de tipo Data Abort, y salta a la rutina ubicada en `0x180`. Se observa cómo se guardan en `R1` los valores `BAD0ADD0` y `00000AB0` que identifican este error.



Lo mismo se observa si se intenta acceder a una dirección fuera de rango. Se puede ver el mismo proceso que en el caso anterior.



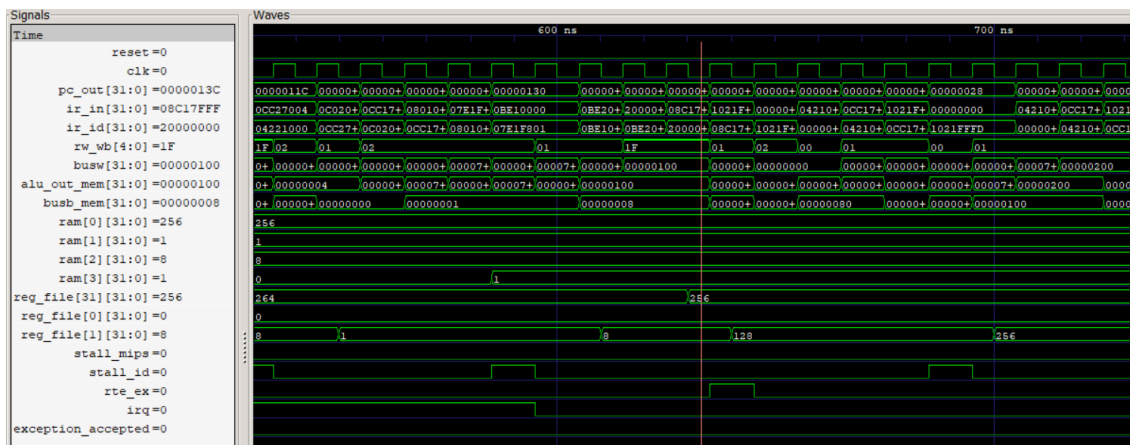
Aquí se intenta ejecutar una instrucción con un código de instrucción desconocido. El PC salta a la dirección 0xC que posteriormente salta a la rutina UNDEF, entrando en un bucle infinito y escribiendo en R1 el valor 0BAD0C0D.

RF7 Retorno a modo usuario RTE

Descripción: Al terminar la excepción se vuelve a la ejecución original con la instrucción RTE.

¿Cómo? Antes de ejecutar RTE, se recupera el valor original de R31 (SP), se carga en el PC la dirección de la instrucción que quedó pendiente cuando la excepción se aceptó y se desactiva el modo excepción.

Verificación: testbench utilizado: Test IRQs:



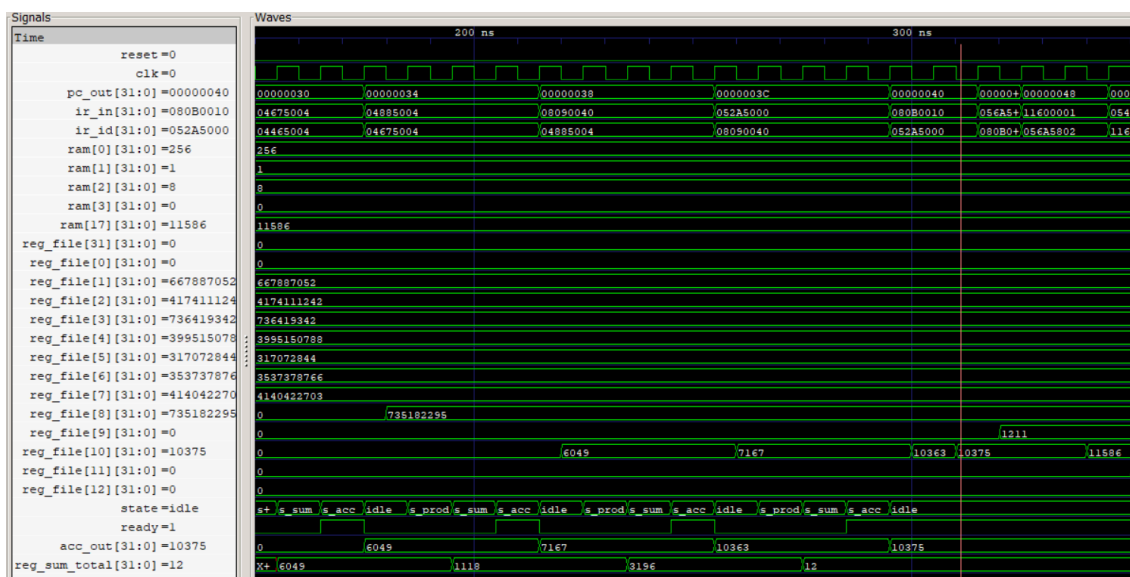
Se puede ver que poco después de llegar al RTE en la señal `ir_id`, `rte_ex` vale '1'. Antes de que esto ocurra, se ha recuperado el valor de R31, volviendo a valer 256, y el PC carga de nuevo una dirección del programa principal, donde R1 sigue multiplicando su valor.

RF8 ALU multiciclo

Descripción: El procesador es capaz de ejecutar las instrucciones MAC y MAC_ini en 3 ciclos y gestionar el estado del registro ACC ante el tratamiento de excepciones.

¿Cómo? Se ha diseñado una ALU multiciclo gestionada por una máquina de estados (FSM) de cuatro estados (IDLE, S_PROD, S_SUM, S_ACC). En el primer ciclo, se realizan cuatro multiplicaciones de 8 bits en paralelo. En el segundo ciclo, se suman los productos parciales, y en el tercero se actualiza el registro acumulador ACC. La señal ready se pone a '0' en estados intermedios para esperar a que finalice el cálculo, y el registro ACC solo se carga mediante la señal load_acc en el último estado para que una posible excepción no interrumpa el cálculo del valor acumulado.

Verificación: testbench utilizado: Delayed SYSTEM MAC (sin nops):



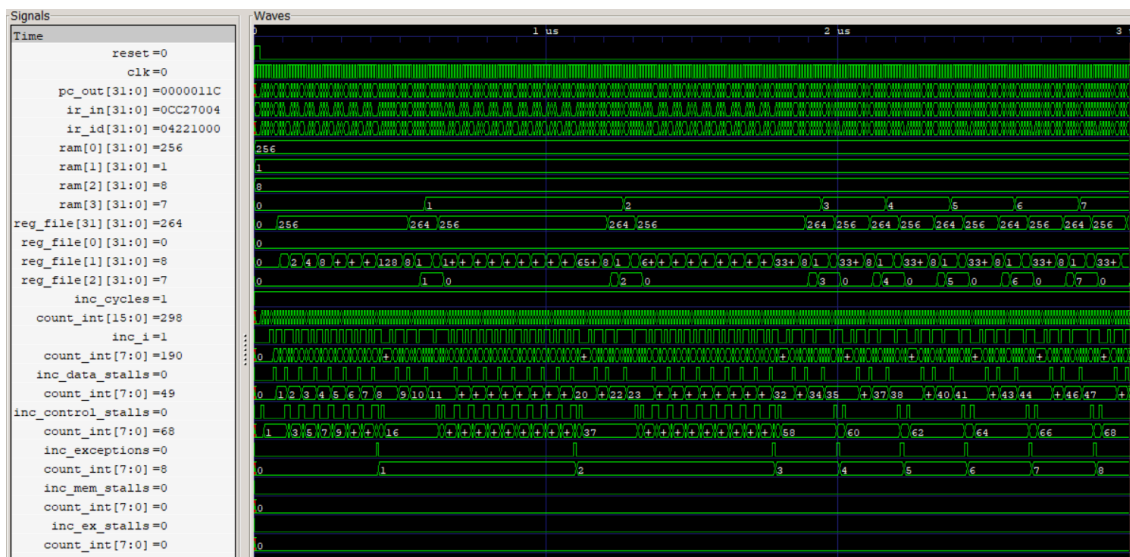
En la señal state se aprecia la transición entre estados, confirmando que la máquina de estados de 3 ciclos se ejecuta correctamente. Mientras el estado no es idle, la señal ready es '0', lo que detiene al procesador el tiempo necesario hasta que la ALU termine el cálculo. Las señal acc_out muestra como el valor acumulado se mantiene estable y solo se actualiza en el estado s_acc. Por último, la señal reg_sum_total permite ver el valor ya sumado antes de integrarse en el acumulador.

RF9 Contadores

Descripción: los contadores nos dan información de la ejecución del código.

¿Cómo? La gestión de los contadores se realiza mediante las señales de incremento que se activan bajo condiciones específicas. Para evitar contar un evento dos veces en el mismo ciclo, se aplica una jerarquía de prioridades en las paradas: memoria, EX, datos, control. El contador de instrucciones solamente se incrementa si la instrucción es válida y ha alcanzado la etapa WB. Las excepciones se contabilizan activando la señal `Exception_accepted`.

Verificación: testbench utilizado: Test IRQs:



Se observa el correcto funcionamiento de los contadores de rendimiento. Destaca la coincidencia entre el contador de excepciones y el registro R2 que las contabilizaba dentro del propio programa. Además, los contadores de datos y control solo incrementan sus valores en ausencia de paradas de mayor prioridad (memoria o EX) y muestran unos valores coherentes, evitando cuentas duplicadas.

Cuantificación de horas dedicadas

- Estudio del MIPS, VHDL, entorno, instalación...: 5
- Adición de las nuevas instrucciones: 4
- Gestión de excepciones: 3
- Gestión de riesgos: 4
- Depuración, verificación y programas de prueba: 25
- Memoria: 8

Conclusiones y autoevaluación

Este proyecto ha consistido en la extensión y optimización de un procesador MIPS de 5 etapas. Entre los hitos alcanzados destacan la transformación de la ALU original en una ALU multiciclo capaz de realizar operaciones MAC vectoriales utilizando una máquina de estados, la implementación de una Unidad de Anticipación y una Unidad de Detención para garantizar la integridad de resultados frente a posibles paradas, la detección y resolución de excepciones para ganar en robustez y el diseño de contadores con jerarquía de prioridades para cuantificar ciclos, instrucciones o ciclos perdidos por diferentes tipos de riesgos.

El proyecto ha sido desafiante pero útil para entender con más precisión el funcionamiento del MIPS. El mayor reto ha sido sin duda la depuración debido a la corrección de errores y las pruebas con distintos bancos de pruebas, pero se ha terminado logrando un correcto funcionamiento y una mejor comprensión de toda la estructura del MIPS.